

The Language That Finally Implements What Academia Invented Forty Years Ago

Why your codebase has invisible privacy bugs, currency mix-ups, and distributed system landmines — and how a compiler can catch them at line one.

There is a gap in software engineering that nobody talks about because everyone has learned to live with it.

On one side: decades of programming language research producing precise, powerful ideas. Information flow types. Units of measure. Typestate protocols. Algebraic operation properties. Linear resource types. On the other side: every production language you've ever shipped. The gap between them is not a secret. It is the normal state of affairs. We have known, for forty years, how to write a compiler that catches currency mix-ups, secret data leaks, and "I called this function twice" bugs — at compile time, in the type system, before the code runs.

Nobody ships it. The ideas stay in papers.

Loom ships it.

The Mars Orbiter Problem, Three Times a Day

On September 23, 1999, the Mars Climate Orbiter was destroyed because one system reported thrust in pound-force seconds and another expected newton-seconds. Both teams implemented their components correctly. The code compiled. The tests passed. The failure was invisible everywhere except at the interface between two systems that had never formally agreed on a unit.

That was one project, \$327 million, one catastrophic failure.

Today, in your codebase right now, there is a function that takes a `Float` named `amount`. Somewhere in that codebase is another function that takes a `Float` named `price`. Both are money. One is USD. One is EUR. They've never met. Someday they will. The compiler will not warn you.

```
fn apply_discount(amount: f64, rate: f64) -> f64 {
    amount * rate // Is amount USD? EUR? A percentage? Who knows.
}
```

In Loom:

```
fn apply_discount :: Float<usd> -> Float<rate> -> Float<usd>
  amount * rate
end
```

The compiler now knows. `Float<usd>` and `Float<eur>` are different types. Adding them is a compile error. `Float<usd> * Float<rate>` produces `Float<usd>`. The Mars Orbiter problem is architecturally unreachable.

```
error[E0001]: unit mismatch
  fn total :: Float<usd> -> Float<eur> -> Float<usd>
    cannot add Float<usd> and Float<eur> - explicit conversion required
```

F# has had this since 2009. One language in forty years.

The Privacy Bug You Didn't Know You Shipped

Every GDPR audit begins the same way: someone asks "which fields contain personal data?" and nobody knows for certain. The answer lives in documentation, in people's heads, in a spreadsheet someone made in 2019 that may or may not be current.

The reason is simple: **type systems have never modeled data sensitivity**. A `String` is a `String` whether it's a username, a social security number, a password, or a debug log message. The compiler treats them identically. The privacy obligations are not in the language. They're in your head.

Here is what Loom's type system looks like after M20:

```
type User =
  id: Int
  email: String @pii @gdpr
  ssn: String @pii @hipaa @encrypt-at-rest
  card_number: String @pci @never-log
  name: String @pii
end
```

The field annotations are not comments. They are compiler inputs. The privacy checker enforces:

- `@pci` without `@encrypt-at-rest` and `@never-log` → compile error
- `@hipaa` without `@encrypt-at-rest` → compile error
- Every emitter carries the semantics forward

The Rust emitter outputs:

```
#[derive(Debug, Clone)]
pub struct User {
    pub id: i64,
    #[loom_pii] #[loom_gdpr]
    pub email: String,
    #[loom_pii] #[loom_hipaa] #[loom_encrypt_at_rest]
    pub ssn: String,
    #[loom_pci] #[loom_never_log]
    pub card_number: String,
}
```

The TypeScript emitter outputs:

```
export interface User {
    id: number;
    /** @pii @gdpr - handle per data protection policy */
    email: string;
    /** @pii @hipaa @encrypt-at-rest */
    ssn: string;
}
```

The OpenAPI emitter outputs:

```
"x-data-protection": {
  "pii-fields": ["User.email", "User.ssn", "User.name"],
  "hipaa-fields": ["User.ssn"],
  "pci-fields": ["User.card_number"]
}
```

Your GDPR audit answer is now: `grep x-pii`. The privacy contract is in the schema, not in someone's head.

The Retry Bug You Shipped to Production

Here is a distributed systems bug so common it has a name: the "double charge." A payment service call times out. The client retries. The server had actually processed the charge the first time. The customer is charged twice.

The fix is idempotency. Everybody knows this. Nobody types it.

```
fn charge_card @idempotent @exactly-once :: PaymentToken -> Amount<usd> ->
Effect<[Payment], Receipt>
  require: amount > 0.0
  token
end
```

`@idempotent` and `@exactly-once` on the same function is a contradiction. The Loom algebraic checker catches it:

```
error[E0004]: contradictory algebraic properties
  fn charge_card: @idempotent and @exactly-once are mutually exclusive
  @idempotent means safe-to-retry; @exactly-once means call exactly one time
```

`@commutative` validates that a function actually has two parameters of compatible types before claiming they can be reordered. `@at-most-once` on a `POST` endpoint forces the OpenAPI emitter to add `"x-retry-policy": "never"` — a machine-readable signal to every API gateway and client library that retrying this call is dangerous.

These properties have been in distributed systems literature since the 1980s. Zero production languages type them. Loom does.

The Protocol Bug That Only Happens in Production

You have a database connection object. You open it, authenticate, query it, close it. Sometimes, in a code path you didn't test, something queries it before authentication. Sometimes it gets queried after close. These bugs are deterministically preventable. The type system just needs to track state.

This is called *typestate*, and it was formally described in 1986. Plaid (Carnegie Mellon, 2009) tried to make it practical. It didn't survive contact with the mainstream. Rust does a version via

ownership, but it requires significant expertise to apply.

In Loom:

```
module Database
  lifecycle Connection :: Disconnected -> Connected -> Authenticated -> Closed

  fn connect      :: String -> Effect<[IO], Connection<Connected>>
  fn authenticate :: Connection<Connected> -> String -> Effect<[IO],
  Connection<Authenticated>]
  fn query        :: Connection<Authenticated> -> String -> Effect<[DB], Rows>
  fn close        :: Connection<Authenticated> -> Effect<[IO], Connection<Closed>]
end
```

The typestate checker validates that every function performing a state transition uses a declared valid transition. Calling `query` on `Connection<Disconnected>` is a type error. Calling `authenticate` on `Connection<Authenticated>` is a type error. The lifecycle is the type.

The Rust emitter generates phantom type structs for each state. The TypeScript emitter generates a state union type. The OpenAPI emitter adds an `x-lifecycle` extension documenting the valid state machine. Every target carries the protocol.

The Security Bug That Type Theory Fixed in 1976

In 1976, Dorothy Denning published "A Lattice Model of Secure Information Flow." The core idea: data has security labels, and a compiler can verify that labeled data doesn't leak across security boundaries without explicit declassification.

Nobody implemented it. JIF (Java Information Flow) was a research compiler from Cornell in 2001. It never shipped. Every language since has treated `password` and `username` as the same type: `String`.

In Loom:

```
module Auth
  flow secret :: Password, Token, SessionKey
  flow tainted :: UserInput, QueryParam
  flow public  :: UserId, Email, Bool
```

```
fn verify :: Password -> UserId -> Effect<[IO], Bool>
  password
end
end
```

The information flow checker knows `Password` is `secret` and `Bool` is `public` . When a `@secret` type flows to a `@public` return in a function that isn't a hash, encrypt, or declassify operation, it's an error:

```
error[E0005]: information flow violation
  fn expose_token: takes @secret Token, returns @public String
  @secret data may not flow to @public outputs
  hint: if this is intentional, use declassify() or rename to indicate exposure
```

The TypeScript emitter generates branded types for labeled types. The OpenAPI emitter adds `x-security-labels` and `x-sensitivity` fields to every schema that carries labeled data. A security audit of your API is now a structural query, not a manual review.

Why No One Shipped This Before

The honest answer is friction. These type systems require the programmer to think in formal terms that are unfamiliar. Units of measure require you to track physical dimensions. Information flow requires you to model a security lattice. Tpestate requires you to reason about program states as a graph. These are not easy things to teach a team.

Two things changed.

First, the emitter can be smarter than the programmer needs to be. Loom infers resource names, REST verbs, path parameters, and error response schemas from your type signatures and function names — without annotations. The same principle applies to semantic properties: the compiler knows enough about your types to help you get the annotations right, and to emit the right artifacts for every target.

Second, the reader changed. An AI assistant with a Loom file and a language specification can produce correct code. It doesn't have to rediscover the invariants from context. The invariants are in the language. They're machine-readable. They survive context boundaries. They're in the schema your AI receives along with the task.

This is the design: not a language for humans to think harder in. A language where the structure carries the intent forward into every artifact the system produces.

What Loom Looks Like End to End

A complete Loom module for a payment service:

```
module PaymentService
  describe: "Handles payment processing with full audit trail"

  flow secret :: CardNumber, CVV, BankToken
  flow tainted :: WebhookPayload

  type Payment =
    id: Int
    amount: Float<usd>
    card_number: String @pci @never-log @encrypt-at-rest
    status: PaymentStatus
  end

  enum PaymentStatus =
    | Pending
    | Completed
    | Failed of String
    | Refunded
  end

  lifecycle Payment :: Pending -> Completed -> Refunded

  fn create_payment @exactly-once :: Float<usd> -> BankToken -> Effect<[Payment],
  Payment]
    require: amount > 0.0
    ensure: result.amount == amount
    amount
  end

  fn refund_payment @idempotent :: Int -> Effect<[Payment], Payment]
    require: payment_id > 0
    payment_id
  end
```

From this single file, Loom materialises:

- **Rust:** Newtype `Usd(f64)` , `#[loom_pci]` on `card_number`, phantom state types, `debug_assert!` on contracts
- **TypeScript:** Branded `type Usd = number & { _unit: "Usd" }` , sensitivity JSDoc, state union type
- **JSON Schema:** `x-unit: "usd"` , `x-pci: true` , `x-encrypt-at-rest: true` , `x-sensitivity: "pci"`
- **OpenAPI 3.0:** Inferred paths `/payments` (POST, `@exactly-once`) and `/payments/{id}` (PUT, `@idempotent`), `x-retry-policy: never` on create, `x-lifecycle` state machine, `x-security-labels` , `x-data-protection` PCI manifest
- **Error:** If you add `@idempotent` to `create_payment` — compile error. If you remove `@encrypt-at-rest` from `card_number` — compile error. If you try to route `BankToken` (secret) to a public return — compile error.

One source file. Five output targets. Every semantic property preserved. Every violation caught before the code runs.

The Language Specification

Loom is open source. The full compiler, test suite, and corpus are at [\[github.com/pragmaworks/loom\]](https://github.com/pragmaworks/loom). The language spec covers:

- Full syntax grammar
- Type system: generics, effects, refinements, units, privacy labels, flow labels, typestate
- Seven GS properties and how Loom enforces each: Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable
- REST inference rules (resource detection, verb inference, path parameters, error schema derivation)
- All five emission targets and their semantic mappings

An AI assistant given the language spec and a task can produce correct Loom. The spec is the mold. The AI is the foundry.

The Double Pyramid: Restriction as the Expansion Mechanism

There is a framing that explains why Loom's type annotations produce *more* useful output rather than less.

Every semantic constraint in Loom removes a degree of freedom from the program space.

`@exactly-once` removes programs where a charge is sent twice. `@pci @never-log` removes programs where card numbers reach log files. `lifecycle Connection :: Closed -> Open` removes programs where a read is issued on a closed connection. These look like restrictions on what you can write. They are. But the space that remains after each restriction is exactly the space of correct programs.

An AI assistant building with Loom does not generate in an unconstrained space and then check. It generates *into* a constrained space and derives correctly on the first pass. The restriction is the expansion mechanism: fewer valid programs, but those are all the right ones. Each checker you add narrows the output space to the subset that is correct — and removes the entire surface of errors that would otherwise require iteration to find.

The compiler is not just finding bugs. It is closing the degrees of freedom that would have required debugging sessions, post-incident reviews, GDPR audit findings, and production outages. Every constraint removed from the programmer's freedom is a failure mode removed from the system's future.

Who Is This For: The Renaissance Practitioner

The standard answer to "who benefits from a richer type system" has historically been: teams building safety-critical software where failure costs lives. The Therac-25 class of failures. Nuclear control systems. Medical devices.

That answer was correct when the cost of maintaining these annotations fell entirely on human practitioners. It is no longer correct.

The engineer who builds and ships alone — a full-stack developer, a data engineer building pipelines, a researcher writing a numerical simulation — has always known that their code has currency bugs, unaudited data paths, and implicit assumptions about resource state. They knew because they wrote the code. The cost of fixing it was never not understanding the problem; it was the annotation burden of making the understanding machine-checkable.

That cost is zero when the AI holds the theory and writes the annotations. The practitioner who works across domains — who understands that their payment pipeline has distributed systems properties, that their API has privacy obligations, that their connection pool has lifecycle constraints — now names those domains in Loom and receives the full formal apparatus of each field applied to their code.

Three properties make this practitioner effective in this environment:

- **Synthetic:** the cross-domain perspective produces insight that neither domain holds alone. The practitioner who sees that a payment API has CRDT properties *and* privacy obligations *and* lifecycle constraints extracts the intersection that specialists in each field would miss.
- **Synaptic:** the connection between domains is where the signal travels. `@exactly-once @idempotent @pci` is not three annotations. It is a complete safety specification for a financial operation, derived from distributed systems, regulatory compliance, and protocol theory simultaneously.
- **Synoptic:** holding multiple domains simultaneously without reducing to any one of them. The .loom file is the synoptic view: units, flows, lifecycle, algebraic properties, all expressed in one coherent specification.

The practitioner who could historically produce synoptic specifications only in prose — in ADRs, in architecture diagrams, in "here's what this code is supposed to do" comments — can now produce them in a form the compiler verifies and the AI derives from, across every output target, from a single source.

The Forty-Year Gap, Closed

The ideas in this article are not new. Units of measure (Kennedy, 1996). Information flow types (Denning, 1976). Typestate (Strom & Yemini, 1986). Algebraic operation properties (distributed systems literature, 1980s). Privacy labels (GDPR, 2018; HIPAA, 1996; PCI-DSS, 2004). These ideas have been waiting for a language that could make them cheap to use.

The cost of a property was always: learn the theory, fight the type system, train your team, maintain the annotations as the code evolves. The benefit was: a class of bug becomes structurally impossible.

Loom's claim is simple: in an AI-native development environment, the cost of a semantic property approaches zero. The AI writes the type signatures. The compiler enforces them. Every output target receives the semantics. The programmer expresses intent. The toolchain handles the rest.

The forty-year gap closes not because the theory got easier. Because the programmer finally has a sufficiently capable reader.

And the reader is capable because it was built by imitating the same brain that produced the theory in the first place.

Loom is built by Pragmaworks. The compiler is written in Rust. The design stems from the Generative Specification methodology described in the GS White Paper (Ghiringhelli, 2026). All features described are implemented in the open-source compiler.